

Darley Technologies - Round 1

The job you are applying for will be fullstack, backend in Rust and frontend with web technologies. If you don't have much experience with Rust and feel that a coding test in Rust wouldn't showcase your full potential, you're welcome to do the backend part in C or C++. If you choose to do so, then make sure to fill out the self-evaluation accordingly, based on the language chosen.

Round 1 consists of two parts, a data structure part in Rust and a frontend part with web technologies. Use your agent of choice for the frontend part. Do not use agents for the backend one. You can do them in parallel or one after another, however you like.

Part 1: Data Structures and Algorithms

Time needed: about 4 hours.

Please document your reasoning in a readme, explaining the steps you took and why you arrived at your chosen solution. You can briefly comment on other solutions and ideas that you had while solving this task.

Do not use agents for this task.

The reviewer will focus on three things: whether the **requirements** are met, the solution is **correct**, and the code **quality** is high.

Create a dataset of words from the book <https://www.gutenberg.org/files/98/98-0.txt>. Implement a fixed-size open addressing hash table by using linear probing to resolve collisions. Assume that the keys are the words from the given dataset and the hash table's values are integers. You need to implement the following functions with a complexity of $O(1)$

- `insert(key, value)`
inserts a new key-value pair or replaces a key's existing value,
- `remove(key)`
removes the corresponding key-value pair,
- `get(key)`
returns the value of the corresponding key,
- `get_last()`
returns the most recent key-value pair that was either inserted or updated and is still present,
- `get_first()`
returns the least recent key-value pair that was either inserted or updated and is still present

Part 2: Orderbook visualization

Feel free to use whichever agent you want for this task, and whichever frontend stack you think fits best, framework or vanilla JS, your call.

Please document in a readme what you focused on, how you used the agent and how you would continue this project if you had more time.

The reviewer will focus on three things: whether the **requirements** are met, how good it looks **visually**, how **performant**/efficient the code is.

Have a look at https://www.binance.com/en/trade/BTC_USDT. Create an SPA that shows an orderbook similar in style to the orderbook on that page. Use at least 20 levels per side (buying, selling). Feel free to use mock data, but it should simulate live updates (e.g. prices changing over time) like it would be when using it from [1].

Bonus points for:

- Using live data [1]
- It being hosted somewhere
- It working on mobile
- Showing a chart

[1]: see <https://developers.binance.com/docs/binance-spot-api-docs/web-socket-streams>

Delivery & Self-Evaluation

On a scale from 1 to 10, rate your overall experience level with each technology you used.

Provide separate ratings for the backend language (Rust, C, or C++) and the frontend stack you chose.

Either upload your solution to an accessible git repository, or — if you use a private GitHub repository — grant access to the user `dt-spr`.